



KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 3 Laboratuvar Föyü

Sınıflar: Kitap Sınıfını İnşa Etmek

Doç. Dr. Tolga Berber · <https://avesis.ktu.edu.tr/tberber>🕒 **Tahmini süre:** 2 saat💻 **Gereksinimler:** C++20 derleyici (g++/clang++), bir metin editörü

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

1 • Ön-öğretim 📣 Neden Önemli?

Geçen hafta bir `struct Kitap` tanımlasaydınız, programınızın herhangi bir satırı `k.yil = -1500;` yazıp nesneyi sessizce geçersiz bir duruma düşürebilirdi — hiçbir şey buna engel olmazdı. Proje büyüyüp onlarca fonksiyon aynı veriye dokundukça, “kim, ne zaman, hangi alanı bozdu?” sorusunun izini sürmek neredeyse imkânsızlaşır. Bu föyde veriyi kimin nasıl değiştirebileceğine artık serbest fonksiyonların değil, tipin kendisinin karar vermesini sağlayacak; kütüphanenizin ilk gerçek sınıfı olan `Kitap`’ı sıfırdan, elinizle inşa edeceksiniz.

1 • Ön-öğretim 🔄 Ön Bilgi

Bu föy YG I ders notu Bölüm 3 (Sınıflar: Kapsülleme ve `Kitap` Sınıfı) üzerine kuruludur ve şu kavramları hazır varsayar: Hafta 2’den C `struct` tanımı ve kullanımı, serbest fonksiyon tanımı/çağırısı, `.hpp/.cpp` ayrımı, `namespace`, `std::string` ve temel I/O (`std::cout`). Sınıf, kurucu ve kapsülleme gibi kavramları bu föyde elinizle kuracağınız için önceden bilmeniz gerekmez; ilgili biçimsel tanımlar ders notu Bölüm 3’ün dört ünitesinde işlenir.

Bu föyde geçen hafta öğrendiğiniz `struct` ve serbest fonksiyon düzenini bir kenara bırakıp, verisini kendi koruyan gerçek bir C++ sınıfı inşa edeceksiniz: kütüphanenizin ilk ve en temel sınıfı olan `Kitap`. Adım adım önce proje iskeletini kuracak, sonra sınıfı bildirecek (`.hpp`), üye fonksiyonlarını tanımlayacak (`.cpp`), ardından bir demo ile derleyip çalıştıracak ve son olarak sınıfın sınırlarını kendiniz test edeceksiniz.

Bu föyü şöyle kullanın: her adımda önce *Ne yapacağız?* kutusundaki görevi tamamlayın (kodu yazın, ardından verilen kabuk komutunu çalıştırın), sonra kendi sonucunuzu *Beklenen* kutusu ve çıktı örneğiyle karşılaştırarak doğru yolda olduğunuzu hemen doğrulayın. Bir şey tutmazsa, sonraki adıma geçmeden o adımı düzeltin — her adım bir öncekinin üstüne inşa edilir. Aradaki *Kontrol Noktası*, o ana kadar yaptıklarınızı bir arada sınamanıza yarar. Bölümler ders notu Bölüm 3’ün üniteleriyle ilerler; her bölümün başında ilgili üniteye bir atıf bulacak, takıldığınızda oraya dönebileceksiniz.

1 Proje İskeletini Kurun

(bkz. Ders notu Bölüm 3, Ünite 4 — Kütüphaneyi Kurmak: Başlık/Kaynak ve İsim Uzayı.)

Adım 1: Klasör ve boş dosyaları oluşturun

❓ Ne yapacağız? kitap-projesi adlı bir klasör ve içinde üç boş dosya (kitap.hpp, kitap.cpp, kitap_demo.cpp) oluşturacak, ardından derleyicinizin C++20 ile çalışmaya hazır olduğunu boş bir derleme denemesiyle doğrulayacaksınız.

❗ Neden? Sınıfın bildirimini (.hpp), tanımını (.cpp) ve kullanımını (kitap_demo.cpp) ayrı dosyalarda tutmak gerçek C++ projelerinin standart yapısıdır — bu ayrımı Ünite 4'te gördüğünüz katalog/raf mantığının kod düzeyindeki karşılığı olarak düşünebilirsiniz. Gerçek kodu yazmaya başlamadan önce boş dosyalarla derleyici zincirinin çalıştığını doğrulamak, ilerideki hataların proje kurulumundan değil kendi kodunuzdan kaynaklandığından emin olmanızı sağlar.

>_ Terminal

```
$ mkdir kitap-projesi && cd kitap-projesi
$ touch kitap.hpp kitap.cpp kitap_demo.cpp
$ g++ -std=c++20 -c kitap.cpp -o kitap.o
```

👁 Beklenen sonuç: Üç dosya boş biçimde oluşur; son komut hiçbir hata veya uyarı vermeden çalışır ve dizinde küçük, boş içerikli bir kitap.o nesne dosyası belirir. Bu, derleyicinizin C++20 standardıyla sorunsuz çalıştığının kanıtıdır — dosyayı silmeniz gerekmez, bir sonraki adımda üzerine yeniden derleyeceksiniz.

2 Sınıfı Tasarlayın

(bkz. Ders notu Bölüm 3, Ünite 1 — Neden Sınıf? Struct'tan Kapsüllemeye; Ünite 2 — Kurucu ve Yıkıcı; Ünite 3 — Kitap Sınıfını İnşa Etmek.)

Adım 2: Sınıfı bildirin (kitap.hpp)

❓ Ne yapacağız? kitap.hpp içine çoklu include korumasını, ad alanını ve Kitap sınıfının tam bildirimini — private alanlar ve public arayüz — yazacaksınız.

❗ Neden? Header dosyası sınıfın kataloğudur: sınıfın dışarıya *ne sunduğunu* ilan eder, *nasıl* yaptığını gizler. #pragma once olmadan bu dosya birden fazla .cpp içinde include edilirse derleyici sınıfı iki kez görür ve "yeniden tanımlama" hatası verir; namespace kitaplik ise dönem boyunca büyüyecek kütüphanenizdeki isim çakışmalarını baştan önler.

```
1 #pragma once
2 #include <string>
3
4 namespace kitaplik {
5
6 class Kitap {
7 public:
8     Kitap(std::string baslik, std::string yazar,
9         std::string isbn, int yil);
10    const std::string& baslik() const { return baslik_; }
11    void setYil(int yil);
12    bool gecerliMi() const;
13    void yazdir() const;
14
15 private:
16    std::string baslik_;
```

Adım 2 (devamı)

```

17     std::string yazar_;
18     std::string isbn_;
19     int yıl_;
20 };
21
22 } // namespace kitaplik

```

Dört alanı da sondaki alt çizgiyle (`baslik_`, `yazar_`, `isbn_`, `yil_`) adlandırın — bu, kod tabanını boyunca "bu bir private üye" anlamına gelen bir kuraldır. `baslik()` getter'ını doğrudan bildirim içinde (satır içi) tanımlanmıştır; diğer dört üye fonksiyonu bir sonraki adımlarda `kitap.cpp` içinde tanımlayacaksınız.

👁 **Beklenen sonuç:** `kitap.hpp` dosyanız artık `Kitap` sınıfının tam bildirimini içerir: dört private alan, bir kurucu bildirimi, bir satır içi getter ve üç ek public üye fonksiyon bildirimi. Dosya henüz tek başına derlenebilir olsa da (bildirimler tanım gerektirmez), bir sonraki adımda eksik tanımları ekleyeceksiniz.

Adım 3: Kurucuyu tanımlayın (`kitap.cpp`)

❓ **Ne yapacağız?** `kitap.cpp` dosyasını açıp `kitap.hpp` başlığını include edecek, ardından parametrelili kurucuyu *üye başlatma listesiyle* tanımlayacaksınız.

📌 **Neden?** Üye başlatma listesi (`:` `baslik_(...)`, `yazar_(...)`, `...`), her alanı nesnenin oluşum anında doğrudan tek adımda ilkler. Bunun yerine gövde içinde atama yapsaydınız (`baslik_ = baslik;` gibi), derleyici önce alanları varsayılan biçimde kurar, sonra üzerlerine yeniden yazardı — gereksiz, iki aşamalı bir işlem. Eğer alanlardan biri `const` ya da referans olsaydı, gövdede atama derleme hatası verirdi; çünkü bu tip üyeler yalnızca ilklenirken değer alabilir, sonradan atanamaz. Bu yüzden üye başlatma listesi yalnız daha verimli değil, bazı durumlarda tek geçerli yoldur.

```

1 #include "kitap.hpp"
2 #include <iostream>
3
4 namespace kitaplik {
5
6     Kitap::Kitap(std::string baslik, std::string yazar,
7                 std::string isbn, int yıl)
8         : baslik_(std::move(baslik)), yazar_(std::move(yazar)),
9           isbn_(std::move(isbn)), yıl_(yıl) {}
10
11 } // namespace kitaplik

```

`#include <iostream>` satırını şimdiden ekleyin — bir sonraki adımda `yazdir()` içinde `std::cout` kullanacaksınız.

👁 **Beklenen sonuç:** `kitap.cpp` dosyanızda ad alanı açık, kurucu üye başlatma listesiyle tanımlanmış durumda. Dosya şu an henüz tek başına derlenmez çünkü `setYıl`, `gecerliMi` ve `yazdir` bildirilmiş ama tanımlanmamış durumda — bunları bir sonraki adımda ekleyeceksiniz.

Adım 4: `gecerliMi()` ve `yazdir()` fonksiyonlarını yazın

❓ **Ne yapacağız?** Aynı `kitap.cpp` dosyasına kalan üç üye fonksiyonu — `setYıl`, `gecerliMi` ve `yazdir` — ekleyeceksiniz.

Adım 4 (devamı)

i Neden? `gecerliMi()`, bir `Kitap` nesnesinin "tutarlı" sayılması için gereken kuralları (başlık boş olmamalı, ISBN 13 karakter olmalı, yıl sıfırdan büyük olmalı) sınıfın *içinde* tutar. Bu doğrulama mantığı sınıf dışında serbest bir fonksiyon olsaydı, her çağırılan kodun bu kuralları hatırlaması ve doğru uygulaması gerekirdi; kapsülleme ilkesi tam olarak bunu önler — kural bir kez, tek yerde yazılır ve her zaman uygulanır.

```

1 bool Kitap::gecerliMi() const {
2     return !baslik_.empty() && isbn_.size() == 13 && yil_ > 0;
3 }
4
5 void Kitap::setYil(int yil) { if (yil > 0) yil_ = yil; }
6
7 void Kitap::yazdir() const {
8     std::cout << baslik_ << " - " << yazar_
9         << " (" << yil_ << "), ISBN: " << isbn_ << '\n';
10 }

```

Üç fonksiyonun da sonundaki `const` anahtar kelimesine dikkat edin: hiçbir nesnenin durumunu *değiştirmez*, yalnız okur veya (`setYil` durumunda) kontrollü biçimde yazar. `setYil` de doğrulama uygular — negatif ya da sıfır bir yıl sessizce reddedilir, alanın tutarsız bir değere düşmesi engellenir.

Ortak Hata / Tuzak

`gecerliMi()` veya `yazdir()` bildiriminin sonundaki `const`'u unutmak yaygın bir hatadır: fonksiyon yine derlenir, ama daha sonra `const Kitap&` alan bir fonksiyona bu nesneyi geçtiğinizde "discards qualifiers" hatası alırsınız. `const` üye fonksiyonu, "bu fonksiyon nesneyi *değiştirmez*" sözünü derleyiciye verir.

👁 Beklenen sonuç: `kitap.hpp` ve `kitap.cpp` artık birlikte tam bir sınıf oluşturur: beş üye fonksiyonun tümü (`kurucu`, `baslik`, `setYil`, `gecerliMi`, `yazdir`) hem bildirilmiş hem tanımlanmış durumdadır.

✓ Kontrol Noktası — devam etmeden doğrulayın

`kitap.hpp` dosyanızda dört `private` alan ve beş `public` bildirim var; `kitap.cpp` dosyanızda `kurucu`, `setYil`, `gecerliMi` ve `yazdir` tanımlanmış durumda. Devam etmeden önce iki dosyayı göz atıp alan adlarının (`baslik_`, `yazar_`, `isbn_`, `yil_`) her ikisinde de birebir aynı yazıldığından emin olun.

3 Derleyin, Çalıştırın, Genişletin

(bkz. *Ders notu Bölüm 3, Ünite 3 — Kitap Sınıfını İnşa Etmek; Ünite 4 — Kütüphaneyi Kurmak: Başlık/Kaynak ve İsim Uzayı.*)

Adım 5: Demo dosyasını yazıp projeyi derleyin

❓ Ne yapacağız? `kitap_demo.cpp` içindeki `main` fonksiyonunda bir `Kitap` nesnesi oluşturacak, `gecerliMi()` ile kontrol edip geçerliyse `yazdir()` çağırarak, ardından tüm projeyi tek komutla derleyip çalıştıracaksınız.

Adım 5 (devamı)

i Neden? Bir sınıfı yalnız yazmak yeterli değildir; gerçek bir nesne üzerinde çalıştığını görmeden tamamlanmış sayılmaz. Bu adım aynı zamanda .hpp/.cpp ayrımının derleme sırasında nasıl bir araya geldiğini de somutlaştırır: kitap_demo.cpp, kitap.hpp sayesinde Kitap sınıfının ne sunduğunu bilir, ama gerçek tanımlar yalnız kitap.cpp derlenip bağlandığında bulunur.

```
1 #include "kitap.hpp"
2
3 int main() {
4     kitaplik::Kitap k{"Nutuk", "Mustafa Kemal",
5                       "9789751000000", 1927};
6     if (k.gecerliMi()) k.yazdir();
7     return 0;
8 }
```

>_ Terminal

```
$ g++ -std=c++20 kitap_demo.cpp kitap.cpp -o kitap_demo
$ ./kitap_demo
```

Windows'ta (PowerShell veya Komut İstemi) derleme komutu aynıdır; yalnız çalıştırma komutu .\kitap_demo.exe biçimindedir.

👁 Beklenen sonuç: Derleme komutu hatasız/uyarısız tamamlanır ve dizinde kitap_demo (Windows'ta kitap_demo.exe) çalıştırılabilir dosyası oluşur. Çalıştırdığınızda ekranda tek satırlık bir çıktı görürsünüz.

🖥 Beklenen Çıktı

Nutuk – Mustafa Kemal (1927), ISBN: 9789751000000

Adım 6: Geçersiz bir kitapla sınırları test edin ve genişletin

? Ne yapacağız? main içine, alanları kasıtlı olarak geçersiz (boş başlık, kısa ISBN, negatif yıl) ikinci bir Kitap nesnesi ekleyecek ve gecerliMi()'nin bunun için false döndüğünü gözlemleyeceksiniz.

i Neden? Doğrulama mantığının işe yaradığını yalnızca "doğru" bir nesneyle görmek yeterli değildir; sınıfın nerede "hayır" dediğini görmek, kapsüllemenin neden değerli olduğunu somutlaştırır — geçersiz veri sessizce kabul edilmez, sınıfın kendisi tarafından yakalanır.

```
1 kitaplik::Kitap gecersiz{"", "Bilinmeyen", "123", -5};
2 std::cout << std::boolalpha << gecersiz.gecerliMi() << '\n';
```

Bu iki satırı kitap_demo.cpp içinde ilk nesnenin altına, return 0; satırından önce ekleyin (<iostream> başlığını da kitap_demo.cpp'nin başına eklemeniz gerekir, çünkü burada doğrudan std::cout kullanıyorsunuz).

>_ Terminal

```
$ g++ -std=c++20 kitap_demo.cpp kitap.cpp -o kitap_demo
$ ./kitap_demo
```

👁 Beklenen sonuç: İlk satırda geçerli kitabın bilgisi, ikinci satırda false yazısı görünür —

Adım 6 (devamı)

üç geçersizlik koşulundan (boş başlık, 13 karakterden kısa ISBN, negatif yıl) herhangi biri tek başına `gecerliMi()`'yi `false` yapmaya yeter.

Beklenen Çıktı

```
Nutuk – Mustafa Kemal (1927), ISBN: 9789751000000
false
```

Görevinizi tamamladıktan sonra, aşağıdakilerden **birini** seçip kendiniz ekleyin:

- `Kitap` sınıfına, `baslik()` ile aynı desende bir `yazar()` getter'ı ekleyin (`kitap.hpp`'de satır içi tanımlı, `const` ve referans döndüren).
- `main` içinde üç `Kitap` nesnesinden oluşan bir `std::vector<kitaplik::Kitap>` (ya da sabit boyutlu bir dizi) oluşturun ve bir `for` döngüsüyle yalnızca `gecerliMi()` olanların `yazdir()`'ini çağırın.

İpucu

Bu haftanın en sık karşılaşılan hataları: (1) `kitap.cpp`'nin başına `#include <iostream>` eklemeyi unutmak — `yazdir()` içinde `std::cout` kullanılamaz hâle gelir; (2) `gecerliMi()` ve `yazdir()` bildirimlerinin sonuna `const` eklemeyi unutmak; (3) kurucu gövdesinde atama yapıp üye başlatma listesini atlamak; (4) derlerken yalnızca `kitap_demo.cpp`'yi verip `kitap.cpp`'yi unutmak — bu, aşağıdaki sorun giderme tablosundaki ilk hatayı üretir; (5) `baslik_` gibi alan adlarının sonundaki alt çizgiyi unutmak veya tutarsız yazmak.

✂ Sorun Giderme

Belirti	Olası neden	Çözüm
undefined reference to <code>`kitaplik::Kitap::...'</code>	<code>kitap.cpp</code> derleme komutuna eklenmedi	<code>g++ ... kitap_demo.cpp kitap.cpp -o kitap_demo</code> biçiminde iki <code>.cpp</code> dosyasını birlikte verin
redefinition of <code>`class kitaplik::Kitap'</code>	<code>#pragma once</code> eksik ya da <code>.hpp</code> elle iki kez include edildi	Dosyanın en başına <code>#pragma once</code> ekleyin
<code>`baslik_'</code> was not declared in this scope	Alan adı yanlış yazıldı ya da yanlışlıkla <code>public</code> bölümüne taşındı	<code>kitap.hpp</code> 'deki alan adlarını <code>kitap.cpp</code> 'deki kullanımla karşılaştırın
passing <code>`const Kitap'</code> as <code>`this'</code> argument discards qualifiers	<code>const</code> bir nesne üzerinde <code>const</code> olmayan bir üye fonksiyon çağırıldı	<code>gecerliMi()</code> ve <code>yazdir()</code> bildirimlerinin sonuna <code>const</code> eklendiğinden emin olun
<code>`cout'</code> was not declared in this scope	<code>kitap.cpp</code> ya da <code>kitap_demo.cpp</code> içinde <code>#include <iostream></code> eksik	İlgili dosyanın başına <code>#include <iostream></code> ekleyin
Çıktıda <code>Kitap</code> nesnesi hiç görünmüyor	<code>gecerliMi()</code> beklenmedik biçimde <code>false</code> döndü, <code>if</code> bloğu atlandı	ISBN'in tam 13 karakter, yılın pozitif ve başlığın boş olmadığını kontrol edin

5 • İzleme ➔ **Aktarım — Başka Nerede?**

Bugün elinizle inşa ettiğiniz beceriler tek bir sınıfa özgü değildir; dönem boyunca ve gerçek projelerde tekrar tekrar karşınıza çıkar:

- **Kitaplık projesine:** “private alan + doğrulayan public arayüz” kalıbı, dönem boyunca eklenecek her sınıfta (bir `Uye`’nin üyelik durumu, bir `Odunc`’un iade tarihi, bir `Raf`’ın kapasitesi) yeniden kullanılır — her tip kendi geçerlilik kurallarını kendi `gecerliMi()`’sinde tutar. `Kitap` için bugün yazdığınız parametrelili kurucu + üye başlatma listesi deseni de bu sınıfların hepsinde aynen tekrarlanır.
- **Başlık/kaynak ayırımına ve isim uzayına:** Sınıfı bildirimi (katalog, `.hpp`) ve tanımını (raf, `.cpp`) olarak ikiye bölüp `namespace kitaplik` içine almak, ölçeklenen her C++ projesinin standart iskeletidir; Kitaplık’a eklenecek her yeni sınıf aynen bu kurulumu tekrarlar ve başka kütüphanelerdeki aynı adlarla çakışmaktan böyle korunur.
- **Kapsülleme fikrine:** “İç durumu gizle, yalnız kontrollü arayüzden eriştir” ilkesi yalnız kodda değil gündelik yaşamda da işler — bir otomat makinesinin para kutusuna elle uzanamaz, yalnız dıştaki tuş/para arayüzüyle etkileşirsiniz; makine kendi stok ve kasa durumunu kendisi yönetir. `Kitap` sınıfınız da `yil_` alanını dışarının doğrudan bozmasına izin vermez, yalnız `setYil()`’in doğrulamasından geçirir.
- **İleriye (Hafta 4):** Bugün yıkıcıya hiç dokunmadınız, çünkü `Kitap` yalnız kendi kendini temizleyen `std::string` üyeler taşıyor. Gelecek hafta bir nesnenin sahip olduğu ham kaynakları (bellek, dosya) güvenle yönetmek için kurucu/yıkıcıyı ve akıllı işaretçileri (`unique_ptr/shared_ptr`) kullanacaksınız — bugünkü sınıf iskeleti o konunun zemini-dir.